

AI Assistant for Coding Help and Meeting Notes

Project Proposal for UCS503P

Sangam Sangwan – CSED

Samyak Jain – CSED

Submitted to: Ms. Paramveer Kaur

Thapar Institute of Engineering and Technology

1 Higher-Order Goal

This project aims to save students time in two ordinary situations: getting stuck on a coding problem, and writing up notes after a meeting. The broader motivation is to reduce small, recurring time losses in day-to-day academic and project work. The immediate engineering objective is narrower and concrete: a small web tool that (a) gives a short hint when a student pastes in an error or a stuck problem, and (b) turns an uploaded meeting recording into a short text summary.

2 Time-to-Value

- **We will build one feature at a time:** the coding-hint feature will be built and tested first, since it is simpler, before starting the meeting-notes feature.
- **We will keep the first version small:** a single working flow (submit a problem, get a hint) is the target for the first iteration, not a finished product.
- **We will avoid unnecessary scope:** features that are not needed to test the core idea (e.g. live audio, browser extensions, editor plug-ins) are left for later and are not promised.

3 Problem Statement

Two small but common problems motivate this project:

- **Coding:** when a student gets an error or gets stuck, they usually search the web or a forum for help. This works, but takes time and often returns answers that don't match their exact code.
- **Meetings:** after a project meeting, someone has to write up what was discussed and what needs to be done. This is often skipped or done incompletely, since it takes extra time after the meeting is already over.

Neither problem is severe, but both are frequent enough that a small tool addressing them is a reasonable, testable engineering project.

4 Proposed Solution

4.1 Overview

A simple web application with two independent features:

- **Coding help:** a text box where a student pastes an error message or describes what they're stuck on, and receives a short hint (not a full solution).
- **Meeting notes:** a page where a student uploads an audio recording of a meeting and receives a short text summary with any action items mentioned.

Both features use the same backend and database, but do not depend on each other.

4.2 Core Workflow

Coding help:

- Student submits an error message or a short description of the problem.
- The system returns a short hint, not the full answer.
- Student marks whether the hint was useful.

Meeting notes:

- Student uploads a recorded audio file of a meeting.
- The system transcribes the audio and generates a short summary with any action items it finds.
- Student can view and download the summary.

4.3 Operational Constraints

- The tool only works on content the student explicitly submits (pasted text or an uploaded file) — no background listening or automatic screen/code access.
- Usable on a normal laptop through a browser; no special hardware or installed drivers needed.
- Built using existing, off-the-shelf APIs for language and speech processing, not custom-trained models.

5 Solution Approach

This is an engineering course project, so the focus is on building a working, well-tested tool using existing components, not on research or new algorithms.

5.1 Hint Generation Approach

- Hints are generated by calling an existing large language model API with a prompt that asks for a short hint, not a complete solution.
- This is a deliberate, simple design choice — it keeps the tool useful for learning rather than for skipping the problem — and is implemented with prompt wording and length limits, not new algorithms.

5.2 Web Architecture

A standard 3-tier web architecture:

- **Frontend:** a simple web page for submitting problems and uploading recordings.
- **Backend API:** handles authentication, submissions, and calls to the language/speech APIs.
- **Database:** stores users, submitted problems, hints given, and meeting summaries.

5.3 CI/CD Approach

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features.

6 Evaluation Criteria

6.1 Primary Evaluation Metric

Time-to-hint (TTH): time between a student submitting a problem and receiving a hint.

- **Target:** median TTH $\leq$ 1 minute during pilot.
- **Attribution:** measured from server timestamps (submission vs. hint returned).

6.2 Secondary Evaluation Metrics

- **Hint usefulness:** percentage of hints marked useful by students (simple thumbs up/down).
- **Summary usefulness:** student rating (1–5) of whether the meeting summary captured the discussion accurately.
- **Reliability:** uptime of the staging/production tool during business hours during the pilot.

6.3 Pilot Validation Plan

- Ask 8–10 classmates to use the coding-help feature on real assignment problems over one to two weeks.
- Ask 2–3 project groups to try the meeting-notes feature on an actual meeting recording.
- Collect the ratings above and a few short comments to guide the next iteration.

7 Scalability

1. **Scalable (theoretically):** the backend is stateless and the frontend is decoupled from it, so more users can be served by running more backend instances; database queries for past hints/summaries are indexed and paginated.

2. **Operationally deployable (practically):** the system runs on standard web hosting with a managed database, a containerized backend, and a CI/CD pipeline for staging and production.

8 Technology Stack

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs and pages.
- Managed relational database.
- An existing LLM API for hint and summary generation.
- An existing speech-to-text API for transcribing uploaded recordings.
- Authentication approach (token-based or session-based).
- Test framework for unit/integration tests.
- CI runner and staging deployment target.

These existing components let us focus on the workflow, correctness, and evaluation rather than building new infrastructure.

9 Project Scope and Deliverables

9.1 Initial Deliverable

- Coding-help feature: submit a problem, receive a hint, mark it useful or not.
- Basic user accounts and submission history.
- Basic logging and timestamps for the TTH metric.
- CI: build + backend unit tests + linting.
- CD to staging with a single command or pipeline job.

9.2 Subsequent Deliverables

- Meeting-notes feature: upload a recording, receive a transcript and summary.
- Simple dashboard showing hint usefulness and summary ratings over time.
- Minor refinements based on pilot feedback (e.g. hint length, prompt wording).

10 Risks and Mitigations

- **Over-reliance on hints:** keep hints short and incomplete by design, so the tool supports rather than replaces problem-solving.
- **Data privacy:** only process content the student explicitly submits; do not store audio files longer than needed for transcription; keep summaries private to the uploading student.

- **API cost or downtime:** add basic rate limits per user and a fallback message if the LLM or speech API is unavailable.
- **Low pilot participation:** recruit pilot users from within our own course/project group first, where participation is easier to arrange.
- **Evaluation ambiguity:** define “useful hint” and “accurate summary” as simple rating questions before development begins, so results are easy to compare.

11 Summary

This project proposes a small, two-feature web tool: a coding-hint assistant and a meeting-notes generator, built from existing APIs rather than new research. It follows the course criteria by shipping a narrow first iteration quickly, using CI/CD for safe releases, and evaluating itself with simple, measurable metrics (time-to-hint and usefulness ratings). The scope is deliberately kept small and achievable within a single semester.